

تکلیف خانگی – پروژه‌ی ClosetAI

یکپارچه‌سازی: Loss، موتور آموزش، Generalization، و Residual

دیتاست: Fashion-MNIST

داستان پروژه

شما به‌تازگی به‌عنوان مهندس ML به یک استارت‌آپ فرضی به نام **ClosetAI** پیوسته‌اید. این شرکت می‌خواهد یک سیستم دسته‌بندی خودکار لباس بسازد که عکس‌های ساده‌ی مشتری‌ها را به یکی از ۱۰ دسته‌ی لباس (تی‌شرت، شلوار، پیراهن، کفش، کیف، و...) تبدیل کند. شما قرار است در ۵ مرحله، این سیستم را از صفر تا یک نسخه‌ی قابل‌دفاع بسازید – دقیقاً با همان ابزارهایی که در Session 1، 2، 3 و cross_entropy.ipynb یاد گرفتید.

راه‌اندازی مشترک (این را در ابتدای نوت‌بوک خود اجرا کنید و در همه‌ی تمرین‌ها از آن استفاده کنید):

```
import torch
import torch.nn as nn
from torch.utils.data import DataLoader, Subset
from torchvision import datasets, transforms

DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")

base_transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5, ), (0.5, )),
    # CIFAR توجه: تک‌کاناله است، نه سه‌کاناله مثل
])

train_full = datasets.FashionMNIST(root="data", train=True, download=True,
transform=base_transform)
test_full = datasets.FashionMNIST(root="data", train=False, download=True,
transform=base_transform)

TRAIN_LIMIT = 4000
TEST_LIMIT = 1000
train_dataset = Subset(train_full, range(TRAIN_LIMIT))
test_dataset = Subset(test_full, range(TEST_LIMIT))
```

```
train_loader = DataLoader(train_dataset, batch_size=64, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=128, shuffle=False)

class_names = train_full.classes
print(class_names)
# ['T-shirt/top', 'Trouser', 'Pullover', 'Dress', 'Coat', 'Sandal', 'Shirt',
'Sneaker', 'Bag', 'Ankle boot']

images, labels = next(iter(train_loader))
print(images.shape) # [64, 1, 28, 28] <- توجه: یک کانال، نه سه
```

تمرین ۱ – انتخاب Loss و اثبات خط پایه

مدیر محصول ClosetAI از شما می‌پرسد: «آیا این مسئله را می‌توانیم مثل چند تصمیم بله/خیر مستقل حل کنیم، یا باید کلاس‌ها را رقیب هم در نظر بگیریم؟»

۱.۱ – تصمیم مستند

با ارجاع مستقیم به جدول cross_entropy.ipynb، مشخص کنید کدام Loss را استفاده می‌کنید (BCEWithLogitsLoss یا CrossEntropyLoss) و در ۲-۳ جمله دلیل بیاورید. آیا ممکن است دو تا از این ۱۰ کلاس هم‌زمان روی یک تصویر صادق باشند؟ این سؤال چطور به انتخاب Loss شما کمک می‌کند؟

۱.۲ – مدل خطی خط پایه

یک nn.Sequential بسازید که یک تصویر [1, 28, 28] را Flatten کند و مستقیماً با یک nn.Linear به ۱۰ کلاس نگاشت دهد. آن را به DEVICE منتقل کنید و روی یک batch واقعی از train_loader اجرا کنید.

۱.۳ – اثبات دستی فرمول

دقیقاً مثل تمرین کلاسی، CrossEntropyLoss پایتورچ را با محاسبه‌ی دستی خودتان

میانگین $\rightarrow -\log \rightarrow$ استخراج احتمال کلاس صحیح $\rightarrow \text{softmax}$

مقایسه کنید. این دو عدد باید تا چند رقم اعشار برابر باشند.

۱.۴ – مدل شما هنوز آموزش ندیده. Loss اولیه‌ی آن را با $-\log(1/10)$ مقایسه کنید.

حالا این را جواب دهید: اگر مدیر محصول تصمیم بگیرد کلاس "Shirt" را کلاً از دیتاست حذف کند (یعنی مسئله ۹ کلاسه شود)، بدون اجرای هیچ کدی، Loss اولیه‌ی مورد انتظار مدل تصادفی چقدر تغییر می‌کند؟ عدد جدید را با فرمول حساب کنید و توضیح دهید چرا این تغییر منطقی است.

تمرین ۲ – یک CNN واقعی و یک باگ ظریف‌تر

۲.۱ – طراحی روی کاغذ (توجه به تک‌کاناله‌بودن)

معماری زیر را طراحی کنید

```
Conv2d(1→16, k=3, p=1) → ReLU → MaxPool2d(2) → Conv2d(16→32, k=3, p=1) → ReLU → MaxPool2d(2) → Flatten → Linear.
```

با ورودی $[B, 1, 28, 28]$ (نه 32×32 مثل CIFAR)، روی کاغذ شکل تانسور را بعد از هر لایه حساب کنید و عدد ورودی Linear را به دست آورید. (راهنما: هر $\text{MaxPool2d}(2)$ اندازه‌ی مکانی را دقیقاً نصف می‌کند. $28 \rightarrow 14 \rightarrow 7$)

۲.۲ – کدنویسی

کلاس FashionCNN را طبق طراحی بالا بنویسید (کانال ورودی Conv اول باید 1 باشد، نه 3) موتور آموزش (`train_one_epoch/evaluate`) را دقیقاً طبق الگوی Session 1 خودتان بنویسید و مدل را ۳ epoch آموزش دهید.

۲.۳ – چالش دیباگ (مرتبط به ۲.۱، نه یک باگ جدید و ناشناخته)

هم‌تیمی شما معماری بالا را با یک تغییر کوچک برای یک دیتاست دیگر کپی کرده، ولی فراموش کرده عدد Linear را متناسب با ورودی 28×28 به‌روزرسانی کند:

```
class FashionCNN(nn.Module):
    def __init__(self, num_classes):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(1, 16, kernel_size=3, padding=1), nn.ReLU(),
            nn.MaxPool2d(2),
            nn.Conv2d(16, 32, kernel_size=3, padding=1), nn.ReLU(),
            nn.MaxPool2d(2),
        )
```

```

self.classifier = nn.Sequential(
    nn.Flatten(),
    nn.Linear(32 * 8 * 8, num_classes),  # این عدد از کجا آمده؟
)

def forward(self, x):
    return self.classifier(self.features(x))

```

سؤال: اگر این کد را روی یک batch واقعی از train_loader اجرا کنید، چه خطایی می‌گیرید؟ با تکیه بر محاسبه‌ی دستی خودتان در ۲.۱ (نه حدس)، عدد درست را به جای $32 * 8 * 8$ بگذارید و توضیح دهید عدد ۸ از کجا اشتباه آمده (احتمالاً از یک دیتاست 32×32 مثل CIFAR کپی شده)

تمرین ۳ – طراحی آزمایش Generalization خودتان

۳.۱ – تصمیم augmentation برای لباس (reasoning جدید، نه تکراری)

class_names را نگاه کنید. برای هرکدام از این سه augmentation، تصمیم بگیرید «مناسب»، «نامناسب»، یا «به شرط تنظیم دقیق پارامتر مناسب» است و با چک لیست سه بخشی فایل (حفظ لیبل، واقع گرایی، حفظ اطلاعات) توجیه کنید:

1. RandomHorizontalFlip(p=0.5) _ به طور خاص برای کلاس "Sandal" یا "Sneaker" فکر کنید: آیا فلیپ افقی یک کفش را به کفش دیگری (پای چپ/راست) تبدیل می‌کند؟ آیا این برای مسئله‌ی ما اهمیتی دارد؟

2. یک RandomRotation فرضی با زاویه‌ی ۹۰ درجه برای کلاس "Trouser" (شلوار) چه مشکلی ایجاد می‌کند؟

3. ColorJitter(brightness=0.3) _ چون تصاویر Fashion-MNIST grayscale هستند (نه رنگی مثل CIFAR)، این transform چه تفاوتی با کاربردش روی CIFAR-10 دارد؟ آیا همچنان معنی‌دار است؟

۳.۲ – کدنویسی: پایپ‌لاین augmentation

یک transforms.Compose به نام train_augmentation بسازید که شامل حداقل دو augmentation باشد که در ۳.۱ «مناسب» تشخیص دادید، به علاوه‌ی ToTensor و Normalize (مشابه راه‌اندازی مشترک).

۳.۳ RegularizedCNN – و آزمایش سه‌گانه

دقیقاً طبق الگوی تمرین کلاسی، یک RegularizedCNN با dropout_p و pooling قابل‌تنظیم بسازید (توجه: کانال ورودی اولین Conv باید 1 باشد). سه آزمایش را اجرا کنید

baseline ، dropout (با p دلخواه خودتان)، و augmentation (با pipeline تمرین ۳.۲)

منحنی‌های train/test هرکدام را رسم کنید.

۳.۴ – تفسیر نتایج واقعی

با نتایج واقعی خودتان (نه فرضی)، طبق جدول تفسیر Session 2 مشخص کنید:

کدام یک از سه آزمایش شما بیشترین نشانه‌ی overfitting را نشان داد؟ آیا dropout شکاف train/test را نسبت به baseline کم کرد؟ نتیجه‌ی augmentation با پیش‌بینی چک‌لیست ۳.۱ شما هم‌خوانی داشت یا نه؟

تمرین ۴ – آیا Residual برای این مسئله‌ی کوچک هم کمک می‌کند؟

۴.۱ – تطبیق معماری با ورودی تک‌کاناله

دقیقاً طبق مشخصات BasicResidualBlock که در کلاس ساختید، آن را اینجا استفاده کنید. یک TinyResNetFashion بسازید با:

- توجه: کانال (stem: Conv2d(1→16, k=3, padding=1) → BatchNorm2d → ReLU (3، نه 1 ورودی)
- stage1: دو بلوک (16→16, stride=1)
- stage2: (16→32, stride=2) سپس (32→32, stride=1)
- head: AdaptiveAvgPool2d((1,1)) → Flatten → Linear(32, 10)

(توجه: این بار فقط ۲ stage داریم، نه ۳ مثل تمرین کلاسی – چون Fashion-MNIST از CIFAR ساده‌تر است و دیتاست کوچک‌تری داریم؛ عمداً یک stage کم شده تا مجبور شوید خودتان تصمیم بگیرید عدد ورودی Linear سرhead چند باید باشد).

سؤال: عدد ورودی Linear در head را با ردیابی shape (نه حدس) به دست آورید و توجیه کنید.

۴.۲ – مقایسه‌ی Plain در برابر Residual روی داده‌ی واقعی

یک PlainConvStack مشابه (چهار لایه‌ی Conv+BN+ReLU پشت‌سرهم، بدون هیچ shortcut ، با همان تعداد کانال کلی) بسازید. هر دو مدل (PlainConvStack و TinyResNetFashion) را ۳ epoch آموزش دهید (با موتور آموزش خودتان از تمرین ۲)

۴.۳ – سؤال یکپارچه‌ساز

با نتایج واقعی: آیا مدل residual شما در epoch اول (نه نهایی) Loss کمتری نسبت به plain داشت؟ این را با شهود تمرین کلاسی A2 (مثال اسکالر $x + \text{residual_branch}(x)$) مرتبط کنید – چرا انتظار داریم مدل residual از یک نقطه‌ی «بهتر» شروع کند، حتی قبل از این‌که آموزش زیادی دیده باشد؟

۴.۴ – تشخیص Degradation در یک سناریوی جدید

فرض کنید یک هم‌تیمی گزارش می‌دهد: «من TinyResNetFashion را عمیق‌تر کردم) ۴ stage به جای ۲ (و حالا دقت train از 0.85 به 0.60 افت کرده، دقت test هم از 0.80 به «0.58 طبق تعریف دقیق فایل 3 Session ، آیا این Overfitting است یا Degradation ؟ یک جمله دلیل بیاورید.

تمرین ۵ – چالش نهایی: ممیزی کامل قبل از عرضه

تیم‌لید ClosetAI کد زیر را برای عرضه به شما تحویل داده. این کد دقیقاً ۶ مشکل دارد که هرکدام از یکی از چهار فایل مرجع می‌آید – نه بیشتر، نه کمتر. کار شما:

الف) هر ۶ مشکل را پیدا کنید و دقیقاً بگویید مربوط به کدام فایل/مفهوم است.

ب) یک نسخه‌ی کامل و اصلاح‌شده از کل این قطعه کد بنویسید.

```
final_model = TinyResNetFashion(num_classes=10) # مشکل ۱

train_aug_full = datasets.FashionMNIST(
    root="data", train=True, download=True, transform=train_augmentation
)
leaky_test_loader = DataLoader(
    Subset(train_aug_full, range(TEST_LIMIT)), batch_size=64, shuffle=True
) # مشکل ۲ و ۳ (دو مشکل جدا در همین خط)
```

```

)

optimizer = torch.optim.Adam(final_model.parameters(), lr=1e-3)
criterion = nn.CrossEntropyLoss()

for images, labels in train_loader:
    images = images.to(DEVICE) # مشکل ۴
    probs = torch.softmax(final_model(images), dim=1) # مشکل ۵
    loss = criterion(probs, labels)
    loss.backward()
    optimizer.step() # مشکل ۶
    (چیزی قبل از این خط جا افتاده)

final_model.eval()
with torch.no_grad():
    for images, labels in leaky_test_loader:
        predictions = final_model(images).argmax(dim=1)

```

راهنمای امتیازدهی به خودتان: اگر کمتر از ۶ مشکل پیدا کردید، به تمرین‌های ۱ تا ۴ همین تکلیف برگردید – هر مشکل این کد دقیقاً یکی از سؤالاتی است که در تمرین‌های قبلی حل کردید.

تمرین بونوس (اختیاری، بدون امتیاز اضافه! – فقط برای تثبیت شخصی و مرور خودتان)

اگر تمرین‌های ۱ تا ۵ را کامل کردید و وقت اضافه داشتید: در حداکثر ۵ جمله‌ی ساده (بدون اصطلاح تخصصی مثل «logits» یا «gradient»)، برای یک مدیرعامل **غیرفنی** توضیح دهید چرا نباید تصاویر تست را دستکاری‌شده (augmented) نگه داشت. اگر نتوانید این را ساده توضیح دهید، احتمالاً باید تمرین ۳ را دوباره مرور کنید.